

Sesión 2 — Prueba de concepto en Java con Arquitectura Limpia

1. Propósito de la sesión

En esta sesión se desarrollará una **prueba de concepto (PoC)** del caso CacaoSelva utilizando **Java**.

El objetivo no es construir todavía un sistema completo para producción, sino comprobar que varios componentes pueden trabajar con la misma información y que el diseño mantiene una separación clara de responsabilidades.

La solución aplicará:

- **Java** como lenguaje de programación.
- **Arquitectura Limpia (Clean Architecture)**.
- **Principios SOLID**.
- **Clean Code**.
- Comunicación mediante **HTTP y JSON**.
- Una **API REST**.
- Una aplicación **Desktop**.
- Un proceso automático denominado **Monitor**.
- Un cliente web como actividad complementaria.

Idea central: la lógica del negocio no debe depender de Spring, JavaFX, HTTP, JSON ni de una base de datos. Las tecnologías externas se conectan alrededor del núcleo de la aplicación.

2. Caso práctico: CacaoSelva

CacaoSelva necesita consultar información relacionada con lotes de cacao.

Para esta prueba se utilizarán datos ficticios correspondientes al 17/09/2026.

2.1 Datos iniciales

ID	Socio	Peso	Estado
1	Ana	120,5 kg	Pendiente
2	Luis	80 kg	Liquidado
3	Rosa	95,25 kg	Pendiente

De estos datos podemos obtener:

- **3 lotes en total.**
 - **2 lotes pendientes.**
 - **1 lote liquidado.**
 - **295,75 kg** de peso total.
-

3. Necesidades del caso

El sistema debe permitir que diferentes tipos de aplicaciones consulten la misma información.

3.1 Necesidades principales

1. Una **API** debe exponer la información de los lotes.
 2. Una aplicación **Desktop** debe permitir que un usuario consulte los lotes.
 3. Un **Monitor** debe consultar automáticamente la API cada 10 segundos.
 4. El sistema debe manejar de forma controlada una interrupción temporal de la API.
 5. Como ejercicio complementario, una página web deberá consumir la misma API.
-

4. Requerimientos

4.1 Requerimientos funcionales

Código	Requerimiento	Criterio de aceptación
RF01	El sistema debe permitir obtener todos los lotes.	GET /lotes retorna los 3 lotes.
RF02	El sistema debe permitir consultar un lote mediante su identificador.	GET /lotes/1 retorna el lote de Ana.
RF03	El sistema debe informar cuando un lote no existe.	GET /lotes/999 retorna HTTP 404 .
RF04	El sistema debe rechazar identificadores con formato inválido.	/lotes/abc retorna HTTP 400 .
RF05	El sistema debe rechazar identificadores no permitidos.	/lotes/0 retorna HTTP 400 .
RF06	La aplicación Desktop debe disponer de una opción para consultar los lotes.	El usuario pulsa Consultar y obtiene los datos.
RF07	La aplicación Desktop debe mostrar los lotes en una tabla.	Se visualizan ID, socio, peso y estado.
RF08	La aplicación Desktop debe informar si una consulta está en proceso.	Se muestra Consultando....
RF09	La aplicación Desktop debe informar si la API no está disponible.	Se muestra un mensaje de error controlado.
RF10	El Monitor debe consultar los lotes automáticamente.	No necesita intervención del usuario.
RF11	El Monitor debe repetir la consulta cada 10 segundos.	Se observan ejecuciones periódicas.
RF12	El Monitor debe contar los lotes en estado Pendiente .	Con los datos iniciales retorna 2 .
RF13	El Monitor debe registrar una falla cuando la API no esté disponible.	La falla se informa sin cerrar inesperadamente el proceso.
RF14	El Monitor debe volver a consultar después de una falla temporal.	Al reiniciar la API vuelve a obtener 2 pendientes .
RF15	El Monitor debe poder detenerse de manera controlada.	El proceso termina correctamente cuando se solicita su cierre.
RF16	El cliente web debe consumir el mismo endpoint /lotes .	Muestra los mismos 3 registros.

Código	Requerimiento	Criterio de aceptación
RF17	El cliente web debe diferenciar un error de conexión de una lista vacía.	Una falla no se presenta como 0 lotes .

4.2 Requerimientos de arquitectura

Código	Requerimiento
RA01	La solución debe desarrollarse siguiendo Arquitectura Limpia .
RA02	La lógica de negocio debe estar separada de frameworks, interfaces gráficas y mecanismos de persistencia.
RA03	Las dependencias del código deben apuntar hacia las capas internas.
RA04	El dominio no debe depender de Spring, JavaFX, HTTP, JSON ni de otras tecnologías externas.
RA05	Los casos de uso deben depender de abstracciones y no de implementaciones concretas.
RA06	El acceso a datos debe definirse mediante puertos o interfaces.
RA07	Las implementaciones concretas deben ubicarse en adaptadores externos o infraestructura.
RA08	La API, Desktop y Monitor deben reutilizar conceptos y contratos del núcleo de la aplicación sin duplicar reglas de negocio.
RA09	Las dependencias externas deben poder reemplazarse con el menor impacto posible sobre el dominio y los casos de uso.
RA10	Las capas deben tener responsabilidades claramente diferenciadas.

4.3 Requerimientos de calidad de código

Código	Requerimiento
RC01	El código debe aplicar los principios SOLID .
RC02	Las clases deben tener una responsabilidad claramente identificable.
RC03	Los nombres de clases, métodos y variables deben expresar su intención.
RC04	Los métodos deben ser pequeños y realizar una tarea concreta.
RC05	Debe evitarse la duplicación innecesaria de código.
RC06	Debe evitarse lógica de negocio dentro de controladores o interfaces gráficas.
RC07	Las dependencias deben recibirse mediante inyección de dependencias cuando corresponda.
RC08	Deben utilizarse excepciones específicas para representar errores relevantes de la aplicación.
RC09	Deben evitarse valores mágicos; los valores importantes deben representarse mediante constantes o configuración.
RC10	El código debe ser comprensible sin depender de comentarios para explicar implementaciones confusas.

4.4 Requerimientos técnicos

Para este laboratorio se utilizará la siguiente base tecnológica:

Código	Requerimiento técnico
RT01	El lenguaje de programación será Java .
RT02	La construcción del proyecto se administrará con Maven .
RT03	La API HTTP se implementará con Spring Boot .
RT04	La aplicación Desktop se implementará con JavaFX .
RT05	El Monitor se implementará como un proceso Java independiente.
RT06	La comunicación entre los componentes se realizará mediante HTTP.
RT07	El intercambio de datos utilizará JSON.
RT08	La API se ejecutará para el laboratorio en <code>http://localhost:5080</code> .
RT09	Los clientes Java utilizarán <code>java.net.http.HttpClient</code> para consumir la API.
RT10	Las operaciones que puedan bloquear la interfaz Desktop se ejecutarán de forma asíncrona.
RT11	El Monitor ejecutará una consulta aproximadamente cada 10 segundos.
RT12	Las configuraciones externas, como URL y tiempos de espera, no deben quedar dispersas en el código.
RT13	Los componentes externos deberán conectarse al núcleo mediante adaptadores.
RT14	La comprobación manual de los endpoints de la API se realizará utilizando Postman.
RT15	Las pruebas de Postman deben registrar método, URL, código HTTP y cuerpo de respuesta.

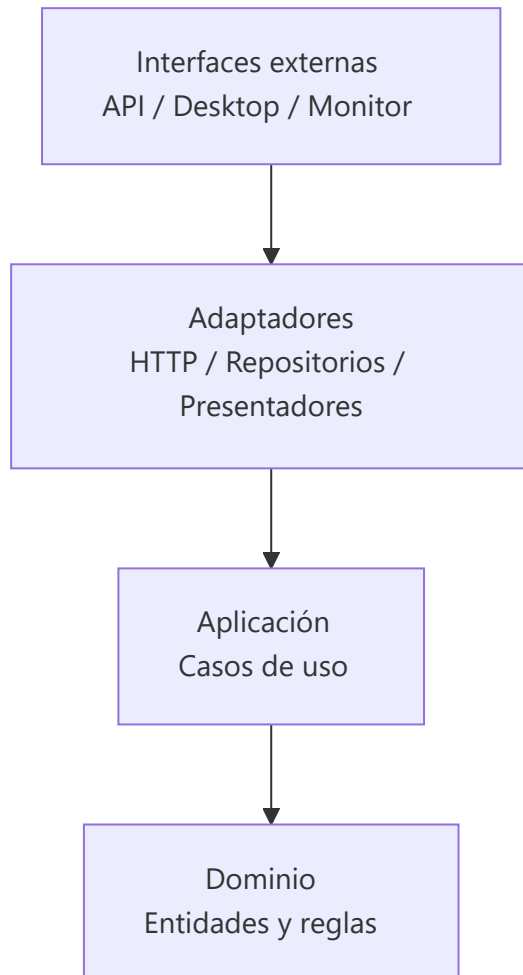
4.5 Restricciones del laboratorio

Código	Restricción
RL01	Los datos se mantendrán inicialmente en memoria.
RL02	No se utilizará base de datos durante esta PoC.
RL03	El ejercicio será de solo lectura.
RL04	No se crearán, actualizarán ni eliminarán lotes.
RL05	Todos los componentes se ejecutarán inicialmente en la misma computadora.
RL06	No se implementará autenticación ni autorización.
RL07	No se realizarán pruebas de carga.
RL08	No puede asegurarse capacidad para 120 usuarios concurrentes.
RL09	No se evaluará alta disponibilidad.
RL10	Esta PoC no representa por sí sola un sistema listo para producción.

5. Arquitectura propuesta

5.1 Idea general

La aplicación se organizará de modo que las reglas importantes estén en el centro.



La regla principal es:

Las capas externas pueden conocer a las internas, pero las internas no deben depender de las externas.

6. Capas de Arquitectura Limpia

6.1 Dominio

Contiene los conceptos principales del negocio.

Para este caso:

```
Lote
EstadoLote
```

Ejemplo:

```
package pe.edu.cacaoselva.domain.model;

import java.math.BigDecimal;

public record Lote(
    Integer id,
    String socio,
    BigDecimal pesoKg,
    EstadoLote estado
) {
}
```

Estado del lote:

```
package pe.edu.cacaoselva.domain.model;

public enum EstadoLote {
    PENDIENTE,
    LIQUIDADO
}
```

Regla

Estas clases **no conocen Spring Boot, JavaFX ni HTTP**.

6.2 Aplicación

Esta capa contiene los **casos de uso**.

Ejemplos:

```
ListarLotes
BuscarLotePorId
ContarLotesPendientes
```

La capa de aplicación responde preguntas como:

¿Qué debe hacer el sistema?

No responde:

7. Puertos

Un puerto es una interfaz que representa algo que la aplicación necesita.

Por ejemplo, los casos de uso necesitan acceder a lotes.

En lugar de depender directamente de una lista, una base de datos o Spring, utilizaremos una interfaz.

```
package pe.edu.cacaoselva.application.port;

import pe.edu.cacaoselva.domain.model.Lote;

import java.util.List;
import java.util.Optional;

public interface LoteRepository {

    List<Lote> findAll();

    Optional<Lote> findById(Integer id);
}
```

El caso de uso conoce esta interfaz, pero **no sabe cómo se obtienen realmente los datos**.

8. Casos de uso

8.1 Listar lotes

```
package pe.edu.cacaoselva.application.usecase;

import pe.edu.cacaoselva.application.port.LoteRepository;
import pe.edu.cacaoselva.domain.model.Lote;

import java.util.List;

public class ListarLotesUseCase {

    private final LoteRepository loteRepository;

    public ListarLotesUseCase(
        LoteRepository loteRepository
    ) {
        this.loteRepository = loteRepository;
    }

    public List<Lote> ejecutar() {
        return loteRepository.findAll();
    }
}
```

Observe que el caso de uso:

- no conoce HTTP;
 - no conoce Spring;
 - no conoce JSON;
 - no conoce JavaFX;
 - únicamente conoce el puerto `LoteRepository`.
-

8.2 Buscar un lote

```
package pe.edu.cacaoselva.application.usecase;

import pe.edu.cacaoselva.application.port.LoteRepository;
import pe.edu.cacaoselva.domain.model.Lote;

public class BuscarLotePorIdUseCase {

    private final LoteRepository loteRepository;

    public BuscarLotePorIdUseCase(
        LoteRepository loteRepository
    ) {
        this.loteRepository = loteRepository;
    }

    public Lote ejecutar(Integer id) {

        if (id == null || id <= 0) {
            throw new IllegalArgumentException(
                "El id debe ser un entero positivo."
            );
        }

        return loteRepository.findById(id)
            .orElseThrow(
                () -> new LoteNoEncontradoException(id)
            );
    }
}
```

Excepción:

```
package pe.edu.cacaoselva.application.exception;

public class LoteNoEncontradoException
    extends RuntimeException {

    public LoteNoEncontradoException(Integer id) {
        super("No existe el lote con id " + id);
    }
}
```

8.3 Contar lotes pendientes

```
package pe.edu.cacaoselva.application.usecase;

import pe.edu.cacaoselva.application.port.LoteRepository;
import pe.edu.cacaoselva.domain.model.EstadoLote;

public class ContarLotesPendientesUseCase {

    private final LoteRepository loteRepository;

    public ContarLotesPendientesUseCase(
        LoteRepository loteRepository
    ) {
        this.loteRepository = loteRepository;
    }

    public long ejecutar() {
        return loteRepository.findAll()
            .stream()
            .filter(
                lote ->
                    lote.estado()
                    == EstadoLote.PENDIENTE
            )
            .count();
    }
}
```

9. Adaptador de datos en memoria

Para esta prueba de concepto no utilizaremos base de datos.

Crearemos una implementación en memoria del puerto `LoteRepository`.

```

package pe.edu.cacaoselva.infrastructure.repository;

import pe.edu.cacaoselva.application.port.LoteRepository;
import pe.edu.cacaoselva.domain.model.EstadoLote;
import pe.edu.cacaoselva.domain.model.Lote;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

public class InMemoryLoteRepository
    implements LoteRepository {

    private final List<Lote> lotes = List.of(
        new Lote(
            1,
            "Ana",
            new BigDecimal("120.5"),
            EstadoLote.PENDIENTE
        ),
        new Lote(
            2,
            "Luis",
            new BigDecimal("80"),
            EstadoLote.LIQUIDADO
        ),
        new Lote(
            3,
            "Rosa",
            new BigDecimal("95.25"),
            EstadoLote.PENDIENTE
        )
    );

    @Override
    public List<Lote> findAll() {
        return lotes;
    }

    @Override
    public Optional<Lote> findById(Integer id) {
        return lotes.stream()
            .filter(lote -> lote.id().equals(id))
            .findFirst();
    }
}

```

Más adelante esta implementación podría reemplazarse por:

PostgreSQL
MySQL
MongoDB
API externa
archivo

sin modificar los casos de uso.

Este es uno de los beneficios de aplicar **inversión de dependencias**.

10. Componente API

La API será un adaptador de entrada.

Su responsabilidad es:

1. recibir una petición HTTP;
2. obtener los datos de entrada;
3. llamar al caso de uso correspondiente;
4. transformar el resultado en una respuesta HTTP.

No debe contener reglas de negocio.

10.1 Endpoints

Método	Ruta	Resultado
GET	/lotes	Lista de lotes
GET	/lotes/{id}	Lote solicitado
GET	/lotes/999	404 Not Found
GET	/lotes/abc	400 Bad Request
GET	/lotes/0	400 Bad Request

10.2 Controlador

Ejemplo simplificado:

```

package pe.edu.cacaoselva.api.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import pe.edu.cacaoselva.application.usecase.BuscarLotePorIdUseCase;
import pe.edu.cacaoselva.application.usecase.ListarLotesUseCase;
import pe.edu.cacaoselva.domain.model.Lote;

import java.util.List;

@RestController
@RequestMapping("/lotes")
public class LoteController {

    private final ListarLotesUseCase listarLotes;
    private final BuscarLotePorIdUseCase buscarLote;

    public LoteController(
        ListarLotesUseCase listarLotes,
        BuscarLotePorIdUseCase buscarLote
    ) {
        this.listarLotes = listarLotes;
        this.buscarLote = buscarLote;
    }

    @GetMapping
    public ResponseEntity<List<Lote>> listar() {
        return ResponseEntity.ok(
            listarLotes.ejecutar()
        );
    }

    @GetMapping("/{id}")
    public ResponseEntity<Lote> buscar(
        @PathVariable Integer id
    ) {
        return ResponseEntity.ok(
            buscarLote.ejecutar(id)
        );
    }
}

```

El controlador no busca directamente en una lista.

En cambio:

Controller



Caso de uso



Puerto



Adaptador de datos

11. Manejo de errores HTTP

Los errores también deben separarse de la lógica principal.

Por ejemplo:

```

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(
        LoteNoEncontradoException.class
    )
    public ResponseEntity<ErrorResponse>
    loteNoEncontrado(
        LoteNoEncontradoException ex
    ) {
        return ResponseEntity
            .status(404)
            .body(
                new ErrorResponse(
                    ex.getMessage()
                )
            );
    }

    @ExceptionHandler(
        IllegalArgumentException.class
    )
    public ResponseEntity<ErrorResponse>
    entradaInvalida(
        IllegalArgumentException ex
    ) {
        return ResponseEntity
            .badRequest()
            .body(
                new ErrorResponse(
                    ex.getMessage()
                )
            );
    }
}

```

Esto evita llenar el controlador de bloques de validación y manejo de excepciones.

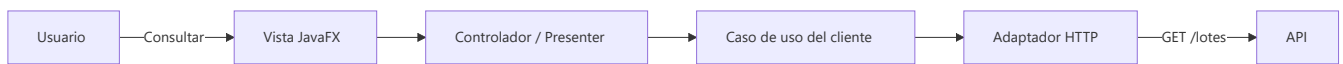
12. Componente Desktop

La aplicación Desktop utilizará JavaFX.

Su función será:

1. recibir una acción del usuario;
2. solicitar los lotes;

3. mostrar el resultado;
4. informar el estado de la operación.



13. Puerto para consumir lotes desde un cliente

El cliente Desktop no debería depender directamente de todos los detalles de HTTP.

Podemos declarar:

```
public interface LoteQueryPort {  
  
    List<LoteDto> obtenerLotes();  
}
```

Y construir un adaptador HTTP:

```
public class HttpLoteQueryAdapter  
    implements LoteQueryPort {  
  
    private final HttpClient httpClient;  
    private final URI endpoint;  
  
    public HttpLoteQueryAdapter(  
        HttpClient httpClient,  
        URI endpoint  
    ) {  
        this.httpClient = httpClient;  
        this.endpoint = endpoint;  
    }  
  
    @Override  
    public List<LoteDto> obtenerLotes() {  
        // Construir petición.  
        // Ejecutar petición.  
        // Validar código HTTP.  
        // Convertir JSON.  
        // Retornar resultado.  
        return List.of();  
    }  
}
```

La lógica relacionada con HTTP queda dentro del adaptador.

La interfaz gráfica no debería construir peticiones HTTP directamente.

14. Flujo de Desktop

Cuando el usuario pulse **Consultar**:

```
Usuario
  ↓
Vista JavaFX
  ↓
Controlador
  ↓
Caso de uso
  ↓
Puerto
  ↓
Adaptador HTTP
  ↓
API
```

Si todo funciona:

```
3 lotes recibidos
```

Si la API está caída:

```
No se pudo conectar con la API
```

15. Evitar bloquear JavaFX

Una petición HTTP puede tardar.

Por eso no debe ejecutarse una operación bloqueante directamente en el hilo de la interfaz.

Una opción es utilizar `CompletableFuture`.

Ejemplo conceptual:

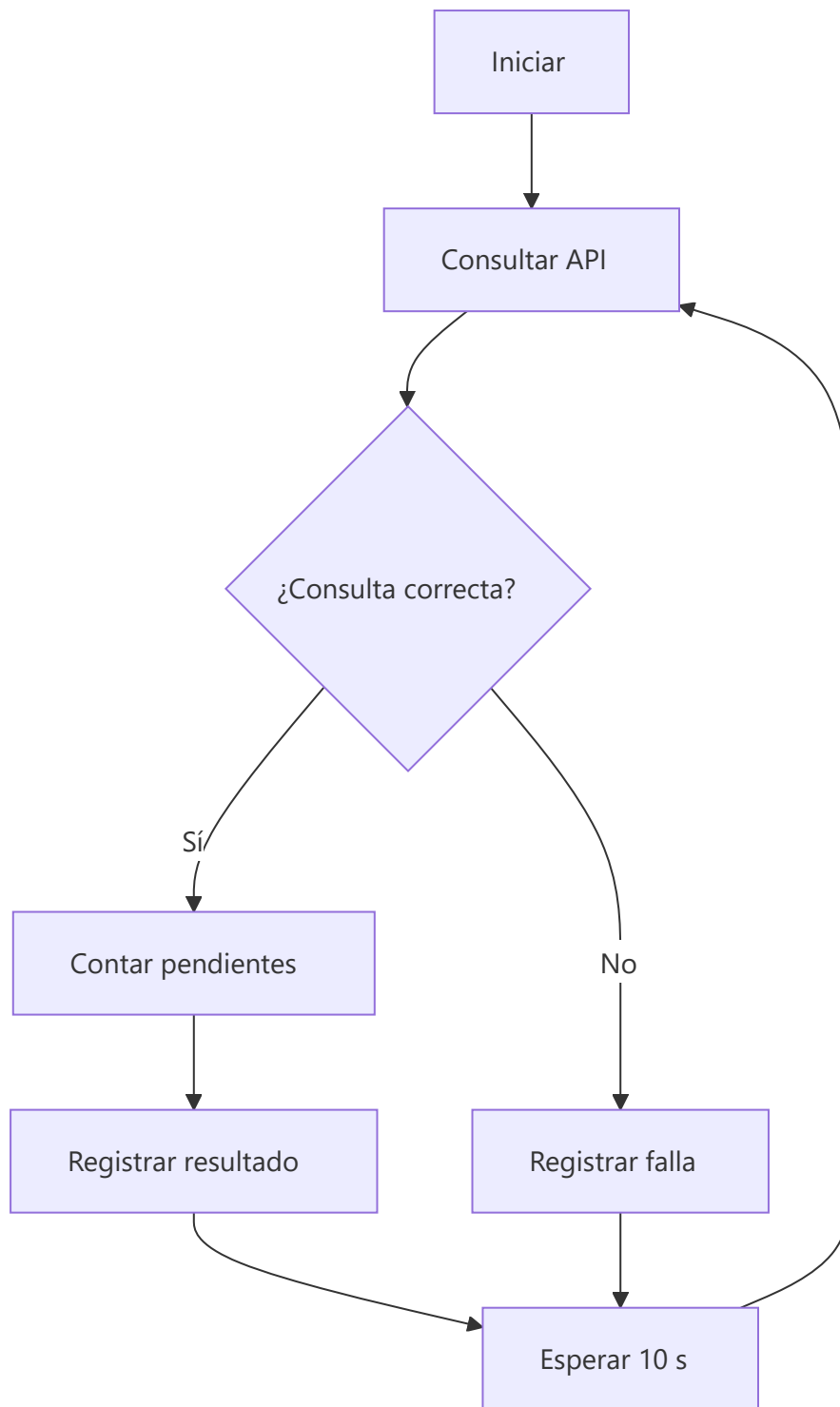
```
CompletableFuture
    .supplyAsync(
        consultarLotesUseCase::ejecutar
    )
    .thenAccept(
        lotes -> Platform.runLater(
            () -> mostrarLotes(lotes)
        )
    )
    .exceptionally(error -> {
        Platform.runLater(
            () -> mostrarError(
                "No se pudo consultar la API"
            )
        );
        return null;
    });
```

Así la ventana puede seguir respondiendo mientras se realiza la consulta.

16. Componente Monitor

El Monitor ejecutará automáticamente una consulta cada 10 segundos.

Su flujo será:



Con los datos iniciales:

Pendientes: 2

17. Monitor y separación de responsabilidades

No debe existir una clase enorme que:

- ejecute el temporizador;
- construya HTTP;
- interprete JSON;
- cuente lotes;
- registre mensajes;
- maneje todos los errores.

Separamos responsabilidades.

Por ejemplo:

```
MonitorScheduler
  ↓
ConsultarLotesUseCase
  ↓
LoteQueryPort
  ↓
HttpLoteQueryAdapter
```

El conteo puede permanecer en un caso de uso específico.

18. Ejecución periódica

Una implementación sencilla puede utilizar:

```
ScheduledExecutorService
```

Ejemplo:

```
ScheduledExecutorService scheduler =
    Executors.newSingleThreadScheduledExecutor();

scheduler.scheduleWithFixedDelay(
    () -> ejecutarMonitoreo(),
    0,
    10,
    TimeUnit.SECONDS
);
```

La tarea periódica no debe contener toda la lógica.

Idealmente:

```
private void ejecutarMonitoreo() {  
  
    try {  
        long pendientes =  
            contarPendientesUseCase.ejecutar();  
  
        logger.info(  
            "Pendientes: {}",  
            pendientes  
        );  
    }  
    catch (Exception ex) {  
        logger.warn(  
            "No fue posible consultar la API"  
        );  
    }  
}
```

19. Recuperación ante una falla

Supongamos que la API está funcionando.

```
Monitor  
↓  
API  
↓  
Pendientes: 2
```

Ahora detenemos la API.

```
Monitor  
↓  
API no disponible  
↓  
Registrar falla  
↓  
Esperar
```

Después reiniciamos la API.

```
Monitor
  ↓
Nueva consulta
  ↓
API disponible
  ↓
Pendientes: 2
```

El Monitor no debe cerrarse simplemente porque una consulta temporal falló.

20. ¿Por qué cuatro consultas no significan ocho pendientes?

Si aparece:

```
Pendientes: 2
Pendientes: 2
Pendientes: 2
Pendientes: 2
```

no existen ocho lotes pendientes.

El Monitor solamente observó cuatro veces el mismo estado del sistema.

Cada consulta es de lectura.

```
Consulta 1 → 2 pendientes
Consulta 2 → 2 pendientes
Consulta 3 → 2 pendientes
Consulta 4 → 2 pendientes
```

El resultado sigue siendo:

```
2 pendientes en cada observación
```

21. Aplicación de SOLID

21.1 S — Single Responsibility Principle

Una clase debe tener una sola razón para cambiar.

Ejemplos:

```
LoteController
```

se encarga de adaptar HTTP.

```
ListarLotesUseCase
```

se encarga de listar lotes.

```
InMemoryLoteRepository
```

se encarga de proporcionar datos en memoria.

No debería existir:

```
LoteManagerQueHaceTodo.java
```

21.2 O — Open/Closed Principle

El software debe poder extenderse sin modificar innecesariamente código estable.

Hoy utilizamos:

```
InMemoryLoteRepository
```

Mañana podríamos implementar:

```
PostgresLoteRepository
```

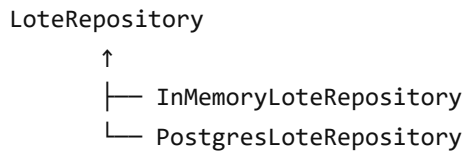
sin modificar:

```
ListarLotesUseCase
```

21.3 L — Liskov Substitution Principle

Una implementación debe poder sustituir a la abstracción que implementa sin romper el comportamiento esperado.

Ejemplo:



El caso de uso debe poder trabajar con cualquiera de las dos.

21.4 I — Interface Segregation Principle

Es preferible utilizar interfaces pequeñas y específicas.

Evitar:

```
interface SistemaRepository {
    buscarLotes();
    guardarUsuario();
    generarReporte();
    enviarCorreo();
    procesarPago();
}
```

Preferir interfaces orientadas a necesidades concretas.

```
LoteRepository
UsuarioRepository
ReportePort
NotificacionPort
```

21.5 D — Dependency Inversion Principle

Los casos de uso no deben depender directamente de implementaciones externas.

Incorrecto:

```
graph TD; ListarLotesUseCase --> PostgreSQL;
```

```
ListarLotesUseCase
    ↓
PostgreSQL
```

Mejor:

```
ListarLotesUseCase
    ↓
LoteRepository
    ↑
PostgresLoteRepository
```

22. Aplicación de Clean Code

22.1 Nombres expresivos

Evitar:

```
var x = repo.f();
```

Preferir:

```
List<Lote> lotes =
    loteRepository.findAll();
```

22.2 Métodos pequeños

Evitar métodos que:

- validen;
- consulten;
- conviertan JSON;
- actualicen la interfaz;
- escriban logs;
- manejen excepciones;

todo al mismo tiempo.

Cada método debe expresar una intención clara.

22.3 Evitar comentarios innecesarios

Incorrecto:

```
// incrementa i
i++;
```

Los comentarios son útiles para explicar:

- una decisión importante;
- una limitación;
- una razón no evidente.

No deben compensar nombres confusos o métodos excesivamente complejos.

22.4 Evitar valores mágicos

Evitar:

```
Thread.sleep(10000);
```

Preferir:

```
private static final Duration  
    INTERVALO_MONITOREO =  
        Duration.ofSeconds(10);
```

22.5 Manejo claro de errores

Evitar:

```
catch (Exception e) {  
}
```

Preferir:

```
catch (ApiNoDisponibleException ex){  
    logger.warn(  
        "API no disponible"  
    );  
}
```

23. Estructura propuesta del proyecto

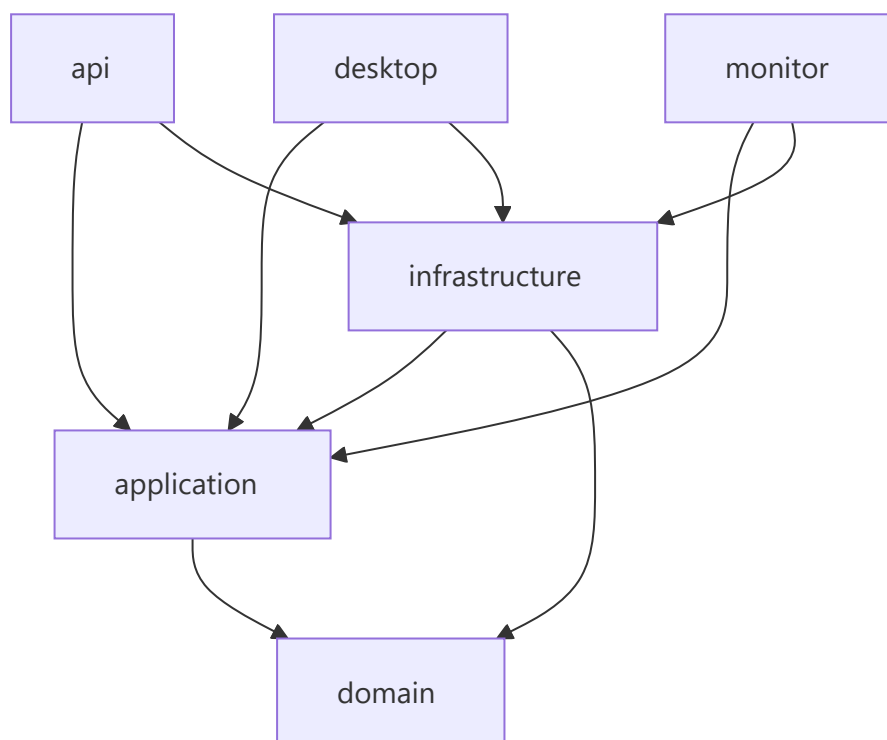
Para fines académicos, la solución puede organizarse como un proyecto Maven multimódulo.

```
cacaoselva/
|
├─ pom.xml
|
├─ domain/
|   └─ src/main/java/
|       └─ pe/edu/cacaoselva/domain/
|           └─ model/
|               ├── Lote.java
|               └─ EstadoLote.java
|
├─ application/
|   └─ src/main/java/
|       └─ pe/edu/cacaoselva/application/
|           ├── port/
|           │   ├── LoteRepository.java
|           │   └─ LoteQueryPort.java
|           ├── usecase/
|           │   ├── ListarLotesUseCase.java
|           │   ├── BuscarLotePorIdUseCase.java
|           │   └─ ContarLotesPendientesUseCase.java
|           └─ exception/
|               └─ LoteNoEncontradoException.java
|
├─ infrastructure/
|   └─ src/main/java/
|       └─ pe/edu/cacaoselva/infrastructure/
|           ├── repository/
|           │   └─ InMemoryLoteRepository.java
|           └─ http/
|               └─ HttpLoteQueryAdapter.java
|
├─ api/
|   └─ src/main/java/
|       └─ pe/edu/cacaoselva/api/
|           ├── controller/
|           │   └─ LoteController.java
|           ├── error/
|           │   └─ ApiExceptionHandler.java
|           └─ CacaoSelvaApiApplication.java
|
├─ desktop/
|   └─ src/main/java/
|       └─ pe/edu/cacaoselva/desktop/
|           ├── view/
|           └─ controller/
```

```
|
|
|
└─ CacaoSelvaDesktopApplication.java
|
|
└─ monitor/
    └─ src/main/java/
        └─ pe/edu/cacaoselva/monitor/
            └─ scheduler/
                └─ MonitorScheduler.java
            └─ CacaoSelvaMonitorApplication.java
```

24. Dependencias entre módulos

La dependencia debe seguir esta dirección:



Lo importante es que:

domain

no dependa de:

api
desktop
monitor
infrastructure
Spring
JavaFX

25. Comprobación con Postman

Para verificar el funcionamiento de la API se utilizará **Postman**.

Postman permitirá comprobar de forma visual:

- método HTTP utilizado;
- URL del endpoint;
- código de estado;
- tiempo de respuesta;
- encabezados;
- cuerpo JSON;
- comportamiento cuando la API no está disponible.

Para esta práctica se recomienda crear una colección denominada:

```
CacaoSelva - Pruebas API
```

y definir una variable:

```
baseUrl = http://localhost:5080
```

De esta forma, las peticiones utilizarán:

```
{{baseUrl}}/lotes
```

en lugar de repetir la dirección completa.

25.1 P01 — Obtener todos los lotes

En Postman:

```
Método: GET  
URL: {{baseUrl}}/lotes
```

Resultado esperado:

```
HTTP 200 OK
```

El cuerpo debe contener tres lotes.

Ejemplo esperado:

```
[
  {
    "id": 1,
    "socio": "Ana",
    "pesoKg": 120.5,
    "estado": "PENDIENTE"
  },
  {
    "id": 2,
    "socio": "Luis",
    "pesoKg": 80,
    "estado": "LIQUIDADO"
  },
  {
    "id": 3,
    "socio": "Rosa",
    "pesoKg": 95.25,
    "estado": "PENDIENTE"
  }
]
```

Evidencia a registrar

El estudiante debe guardar:

- captura de la petición;
- código 200;
- cuerpo JSON;
- fecha y hora de la prueba.

25.2 P02 — Obtener un lote por ID

En Postman:

```
Método: GET
URL: {{baseUrl}}/lotes/1
```

Resultado esperado:

```
HTTP 200 OK
```

El cuerpo debe corresponder al lote de Ana:

```
{
  "id": 1,
  "socio": "Ana",
  "pesoKg": 120.5,
  "estado": "PENDIENTE"
}
```

25.3 P03 — Consultar un lote inexistente

En Postman:

```
Método: GET
URL: {{baseUrl}}/lotes/999
```

Resultado esperado:

```
HTTP 404 Not Found
```

El cuerpo debe contener un mensaje de error.

Ejemplo:

```
{
  "message": "No existe el lote con id 999"
}
```

La finalidad de esta prueba es demostrar que:

un identificador válido no implica que el recurso exista.

25.4 P04 — Identificador inválido

Caso A

```
Método: GET
URL: {{baseUrl}}/lotes/abc
```

Resultado esperado:

```
HTTP 400 Bad Request
```

Caso B

Método: GET

URL: {{baseUrl}}/lotes/0

Resultado esperado:

HTTP 400 Bad Request

Estas pruebas permiten distinguir entre:

400 → solicitud inválida

404 → recurso válido pero inexistente

25.5 Pruebas automáticas básicas en Postman

Postman también puede ejecutar pequeñas verificaciones automáticas.

En la pestaña **Tests** de PO1 puede agregarse:

```
pm.test("La respuesta es 200", function () {
  pm.response.to.have.status(200);
});

pm.test("La respuesta contiene 3 lotes", function () {
  const datos = pm.response.json();
  pm.expect(datos).to.be.an("array");
  pm.expect(datos.length).to.eql(3);
});

pm.test("Existen 2 lotes pendientes", function () {
  const datos = pm.response.json();

  const pendientes = datos.filter(
    lote => lote.estado === "PENDIENTE"
  );

  pm.expect(pendientes.length).to.eql(2);
});
```

Para PO3:

```
pm.test("Lote inexistente retorna 404", function () {  
    pm.response.to.have.status(404);  
});
```

Para P04:

```
pm.test("Identificador inválido retorna 400", function () {  
    pm.response.to.have.status(400);  
});
```

25.6 Organización sugerida de la colección

```
CacaoSelva - Pruebas API  
|  
├─ P01 - Listar lotes  
├─ P02 - Buscar lote 1  
├─ P03 - Lote inexistente  
├─ P04A - ID no numérico  
├─ P04B - ID cero  
└─ P07 - API no disponible
```

25.7 P05 — Desktop

Acción:

Pulsar Consultar

Esperado:

3 filas

con:

Ana
Luis
Rosa

P06 — Monitor

Esperado:

Pendientes: 2

durante varios ciclos.

P07 — Interrupción de la API

Esta prueba combinará **Postman**, **Desktop** y **Monitor**.

Procedimiento

1. Iniciar la API.
2. Ejecutar en Postman:

```
GET {{baseUrl}}/lotes
```

3. Verificar que Postman responda:

```
200 OK
```

4. Iniciar Desktop.
5. Iniciar Monitor.
6. Detener únicamente la API.
7. Volver a ejecutar en Postman:

```
GET {{baseUrl}}/lotes
```

8. Registrar el error de conexión mostrado por Postman.
9. Consultar desde Desktop.
10. Observar el comportamiento del Monitor.
11. Reiniciar la API.
12. Ejecutar nuevamente la petición desde Postman.
13. Esperar el siguiente ciclo del Monitor.

Resultado esperado

Antes de la falla:

Postman → 200 OK

Desktop → 3 lotes

Monitor → Pendientes: 2

Durante la falla:

Postman → error de conexión

Desktop → informa que la API no está disponible

Monitor → registra la falla y continúa activo

Después de reiniciar:

Postman → 200 OK

Monitor → vuelve a mostrar Pendientes: 2

Evidencia

Guardar:

- captura de Postman antes de detener la API;
- captura del error de conexión;
- captura de la respuesta después de reiniciarla;
- registro del Monitor;
- captura del mensaje mostrado por Desktop.

P08 — Detención del Monitor

Detener voluntariamente el proceso.

Esperado:

el Monitor termina de forma controlada

26. Matriz de trazabilidad

Prueba	Requerimientos relacionados
Po1	RF01, RA02, RT06, RT07
Po2	RF02
Po3	RF03
Po4	RF04, RF05
Po5	RF06, RF07, RF08, RF09
Po6	RF10, RF11, RF12
Po7	RF09, RF13, RF14
Po8	RF15
Arquitectura	RA01–RA10
Calidad del código	RC01–RC10

27. Taller

Consigna

En equipos, construyan una prueba de concepto que implemente:

- 1. dominio;
- 2. casos de uso;
- 3. puertos;
- 4. adaptador de datos en memoria;
- 5. API;
- 6. cliente Desktop;
- 7. Monitor.

Después deberán demostrar el funcionamiento utilizando las pruebas PO1–PO8.

28. Orden recomendado de implementación

Paso 1 — Dominio

Construir:

Lote
EstadoLote

Paso 2 — Puertos

Construir:

LoteRepository

y los puertos que necesiten los clientes.

Paso 3 — Casos de uso

Implementar:

ListarLotesUseCase
BuscarLotePorIdUseCase
ContarLotesPendientesUseCase

Paso 4 — Repositorio en memoria

Implementar:

InMemoryLoteRepository

Paso 5 — API

Construir:

LoteController
ApiExceptionHandler

Paso 6 — Desktop

Construir:

Vista JavaFX
Controlador
Adaptador HTTP

Paso 7 — Monitor

Construir:

MonitorScheduler
Adaptador HTTP

Paso 8 — Configurar Postman

Crear la colección:

CacaoSelva - Pruebas API

Agregar las peticiones PO1–PO4 y configurar:

baseUrl = http://localhost:5080

Paso 9 — Pruebas

Ejecutar PO1–PO8.

Las pruebas de API deben verificarse y documentarse desde Postman.

29. Preguntas para revisar la arquitectura

El estudiante debe poder responder:

1. ¿Dónde se encuentra la entidad `Lote` ?
2. ¿Por qué `Lote` no debe depender de Spring?
3. ¿Cuál es la responsabilidad de `LoteController` ?
4. ¿Dónde se encuentra la regla para buscar un lote?
5. ¿Qué es un puerto?
6. ¿Qué es un adaptador?
7. ¿Por qué `InMemoryLoteRepository` implementa una interfaz?
8. ¿Cómo podría reemplazarse el almacenamiento en memoria por PostgreSQL?
9. ¿Qué principio SOLID facilita ese reemplazo?

10. ¿Por qué la interfaz JavaFX no debe realizar directamente toda la lógica HTTP y de negocio?

30. Casos para analizar

Caso 1

Un estudiante escribe toda la lógica dentro de:

```
LoteController
```

incluyendo:

- datos;
- validaciones;
- conteos;
- HTTP;
- reglas de negocio.

Pregunta

¿Qué problema existe?

Respuesta esperada

Se mezclan responsabilidades.

El controlador debería ocuparse principalmente de adaptar HTTP y delegar la lógica a los casos de uso.

Se incumple especialmente el principio de responsabilidad única.

Caso 2

ListarLotesUseCase contiene:

```
@Autowired  
private JdbcTemplate jdbcTemplate;
```

Pregunta

¿Por qué contradice la Arquitectura Limpia?

Respuesta esperada

El caso de uso queda acoplado directamente a una tecnología de infraestructura.

Debe depender de un puerto como:

LoteRepository

y una implementación externa debe encargarse del acceso concreto.

Caso 3

Se cambia:

InMemoryLoteRepository

por:

PostgresLoteRepository

pero también es necesario modificar todos los casos de uso.

Pregunta

¿Qué señal de diseño aparece?

Respuesta esperada

Los casos de uso probablemente están demasiado acoplados a la implementación concreta del repositorio.

El objetivo es que dependan de la abstracción.

Caso 4

Desktop recibe un error de conexión y muestra:

0 lotes

Pregunta

¿Es correcto?

Respuesta esperada

No.

Una lista vacía obtenida correctamente y una falla de comunicación representan situaciones distintas.

Caso 5

El Monitor imprime:

Pendientes: 2
Pendientes: 2
Pendientes: 2

y el estudiante afirma que existen seis pendientes.

Respuesta esperada

Incorrecto.

Son tres observaciones del mismo estado, no tres conjuntos diferentes.

31. Entregable

El equipo debe entregar:

Elemento	Contenido
Requerimientos	RF, RA, RC y RT utilizados
Diagrama	Arquitectura de la solución
Estructura	Módulos y paquetes
Código	Proyecto Java
Evidencia	Ejecución de Po1–Po8, incluyendo capturas y resultados de Postman
Matriz	Requerimiento → prueba
Conclusiones	Qué se demostró y qué no
Revisión SOLID	Identificar dónde se aplica cada principio

No incluir:

```
target/  
.idea/  
.class
```

en el entregable de código fuente.

32. Criterios de revisión

El proyecto no debe evaluarse únicamente porque “funciona”.

También debe comprobarse:

Funcionalidad

- API responde correctamente.
- Desktop consulta.
- Monitor ejecuta periódicamente.
- Las fallas se controlan.

Arquitectura

- El dominio permanece independiente.
- Los casos de uso están separados.
- Existen puertos.
- Existen adaptadores.
- Las dependencias apuntan hacia el núcleo.

SOLID

- Responsabilidades claras.
- Dependencia de abstracciones.
- Interfaces específicas.
- Implementaciones sustituibles.

Clean Code

- nombres expresivos;
 - métodos pequeños;
 - poca duplicación;
 - manejo explícito de errores;
 - ausencia de valores mágicos innecesarios;
 - clases con propósito claro.
-

33. ¿Qué demuestra esta prueba de concepto?

Si las pruebas son satisfactorias podremos afirmar que:

- Java permite implementar los componentes propuestos;
- API, Desktop y Monitor pueden intercambiar información;
- el mismo contrato puede utilizarse desde diferentes clientes;
- la aplicación puede distinguir éxito y falla;
- el Monitor puede recuperarse después de una interrupción temporal;
- el diseño puede organizarse mediante Arquitectura Limpia;
- las dependencias de infraestructura pueden mantenerse fuera del núcleo;
- se pueden aplicar principios SOLID y Clean Code desde una PoC.

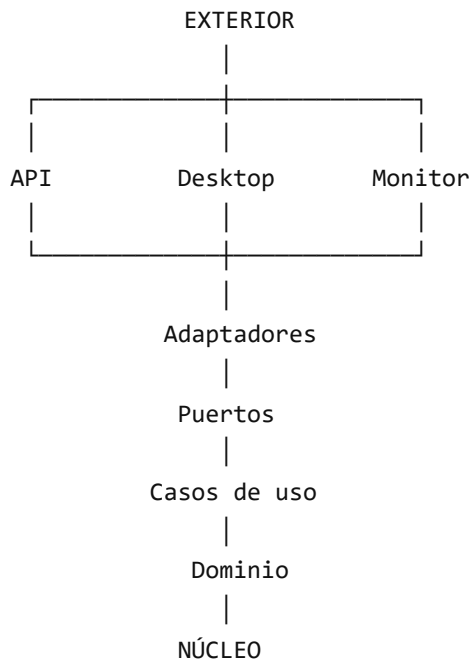
34. ¿Qué NO demuestra?

La prueba no demuestra todavía:

- seguridad;
- autenticación;
- autorización;
- persistencia real;
- transacciones;
- alta disponibilidad;
- rendimiento;
- escalabilidad;
- soporte para 120 usuarios concurrentes;
- despliegue distribuido real;
- preparación completa para producción.

Estas características requerirían nuevos requerimientos y nuevas pruebas.

35. Resumen visual



La dependencia debe mirar hacia el centro:

Tecnología
↓
Adaptadores
↓
Aplicación
↓
Dominio

No al contrario.

36. Idea principal para recordar

Arquitectura Limpia no significa crear muchas carpetas. Significa organizar las dependencias para proteger las reglas de negocio de los detalles tecnológicos.

Y:

SOLID y Clean Code no son requisitos separados de la arquitectura: ayudan a mantener cada responsabilidad clara, reducir el acoplamiento y facilitar cambios futuros.

Referencias base sugeridas

- Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
- Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*.
- Robert C. Martin. *Design Principles and Design Patterns* — principios SOLID.
- Oracle. Documentación oficial de Java.
- Spring. Documentación oficial de Spring Boot.
- OpenJFX. Documentación oficial de JavaFX.
- IETF. RFC 9110: HTTP Semantics.
- Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*.